BACKEND ENGINEERING · HLD INTERVIEW PREP

Monolith vs Microservices

Architecture & Design Patterns

Strangler Pattern · Saga Pattern · CQRS
Decomposition · Database Strategy · Distributed Transactions

TOPICS COVERED

Table of Contents

1. Monolithic Architecture

A **monolithic application** packages all business functionality — orders, payments, inventory, login — into a single deployable unit sharing one database. Often called a "legacy application" in companies undergoing migration.

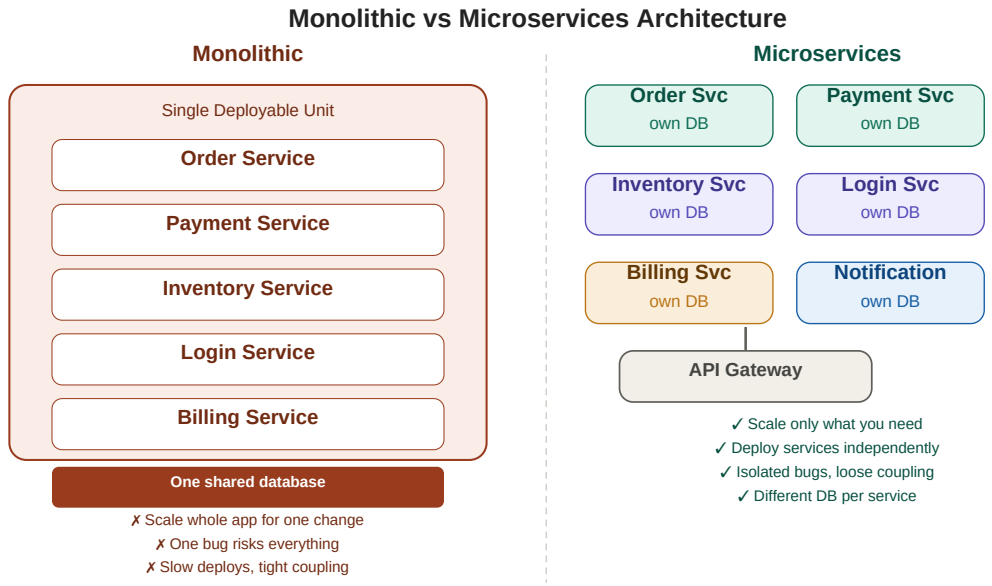
Key disadvantages:

- **Scaling inefficiency** — must replicate the entire app even if only one service needs more resources
- **Tight coupling** — a change to payment code can break order logic; one bug risks everything
- **Slow deployments** — any change requires full regression testing and redeployment
- **Hard to debug** — large interdependent codebase makes tracing failures time-consuming

2. Microservices Architecture

Microservices break a large application into **small, independent services** each responsible for one business capability. Each service has its own database, can be deployed independently, and communicates via well-defined interfaces.

- **Independent scaling** — scale only the Order Service if orders spike, not the whole app
- **Independent deployment** — deploy a bug fix to Payment Service without touching others
- **Fault isolation** — a crash in Notification Service does not bring down Order Service
- **Technology flexibility** — each service can use a different language, framework, or DB type



Monolith — one big unit, one shared DB vs Microservices — independent services each with own DB

3. Monolith vs Microservices — Comparison

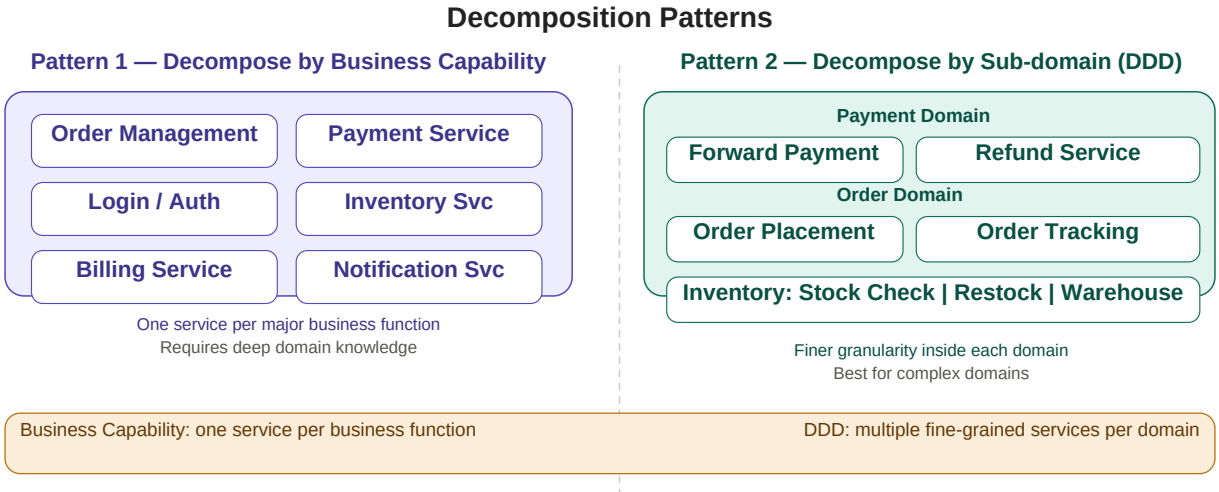
Aspect	Monolithic	Microservices
Deployment	Whole app redeployed on any change	Deploy individual service only
Scaling	Scale entire application	Scale only the service that needs it

Aspect	Monolithic	Microservices
Coupling	Tightly coupled — changes cascade	Loosely coupled via interfaces
Debugging	Hard — one huge codebase	Easier — isolated per service
Database	One shared database	Each service owns its DB
Fault isolation	One bug can crash everything	Failures contained per service
Dev speed	Slows as codebase grows	Teams own services independently
Complexity	Simple initially	Complex distributed system
Transactions	ACID — straightforward rollback	Distributed — needs Saga pattern
Best for	Small apps, early stage startups	Large scale, multiple teams

Microservices introduce distributed system complexity — transaction management, monitoring, and service communication all require careful design.

Proper decomposition is critical: poorly designed microservices with tight dependencies behave like a distributed monolith — worst of both worlds.

4. Decomposition Patterns



Business Capability (left) — one service per function | DDD (right) — fine-grained sub-services within a domain

4.1 Business Capability

Services map directly to **major business functions**. Each function — Order Management, Payment, Inventory, Login — becomes one service. Requires deep domain knowledge to identify correct boundaries. No strict size definition; size depends on project context.

4.2 Domain-Driven Design (DDD)

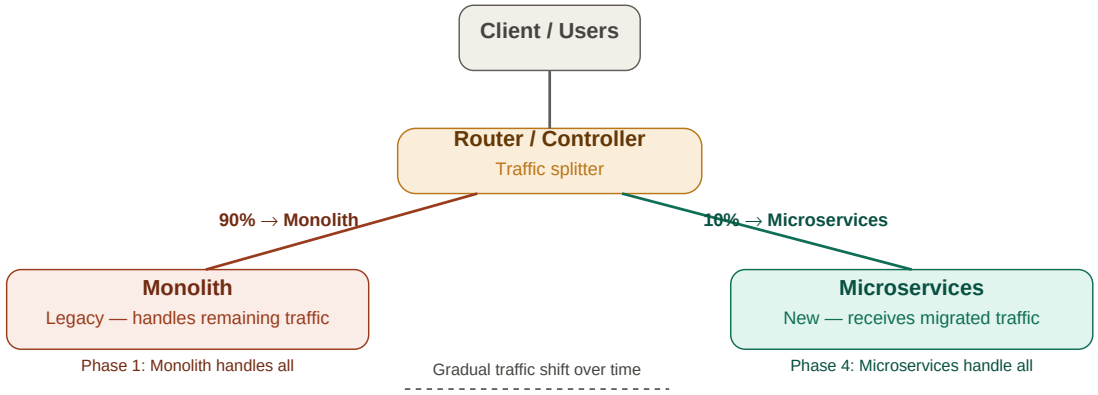
Domains are split into **smaller sub-domains**, each becoming its own service. Provides finer granularity than business capability. Example: Payment domain splits into Forward Payment and Refund Service; Order domain splits into Order Placement and Order Tracking.

Aspect	Business Capability	Sub-domain (DDD)
Service granularity	One per major business function	Multiple per domain
Service size	Relatively larger	Smaller, finer-grained
Example	One Order Service	Order Placement + Order Tracking
Best for	Clear-cut business functions	Complex domains needing deep split
Challenge	Identifying correct boundaries	Understanding domain sub-divisions

5. Strangler Pattern — Gradual Migration

The Strangler Pattern migrates a monolith to microservices **incrementally** — never an all-or-nothing rewrite. A router sits in front of the system, splitting traffic between the old monolith and new microservices. As confidence grows, traffic shifts until the monolith is fully replaced.

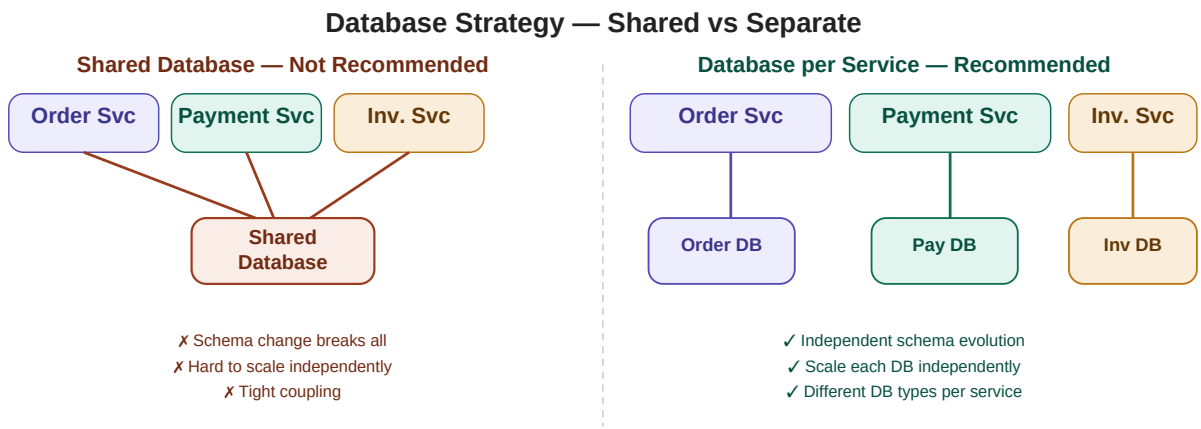
Strangler Pattern — Gradual Migration



Router splits traffic — starts 90/10, gradually shifts to 100% microservices over time

- Start with 10% traffic to microservices — test in production with real but limited load
- Instantly redirect back to monolith if the new service fails or has bugs
- Gradually increase microservice share as stability and confidence grows
- Monolith deprecated only when 100% traffic is successfully handled by microservices

6. Database Strategy — Shared vs Separate



Shared DB (not recommended) — tight coupling | Separate DB per service (standard practice)

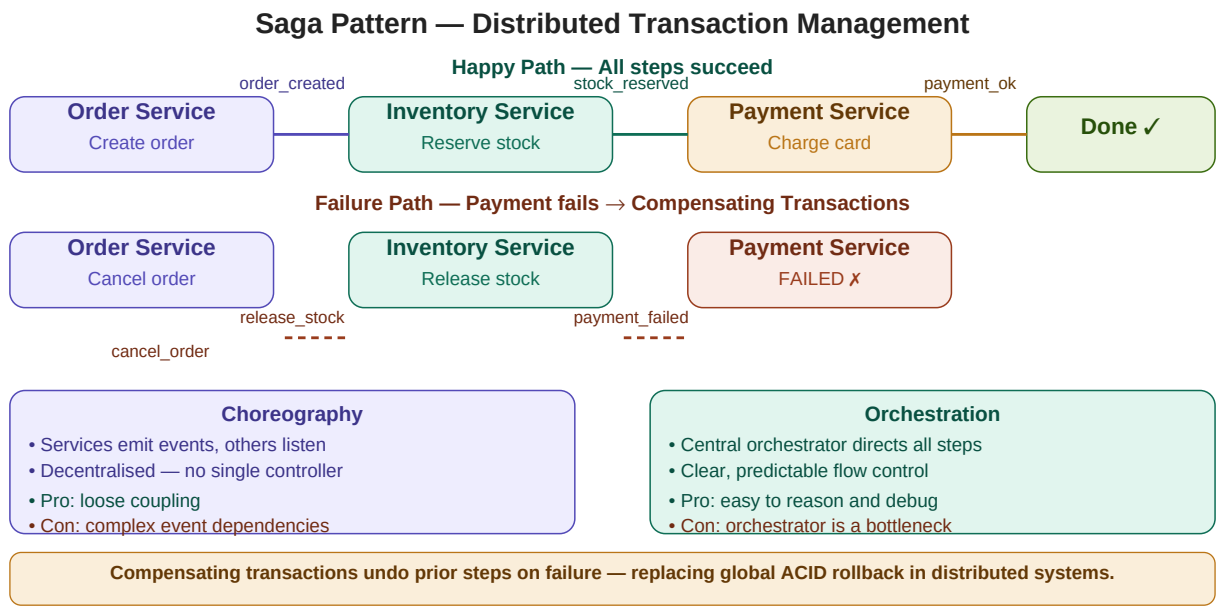
Standard practice: each microservice owns its own database. No other service can directly access it.

Trade-off: cross-service joins become harder — solved via CQRS (read views) or the Saga pattern for transactions.

Benefit: services can evolve schemas independently, choose different DB types, and scale their DB separately.

7. Saga Pattern — Distributed Transaction Management

When each microservice has its own DB, there is no global ACID transaction spanning all services. The **Saga Pattern** manages this by breaking a transaction into a sequence of **local transactions** coordinated via events. On failure, **compensating transactions** reverse prior steps.

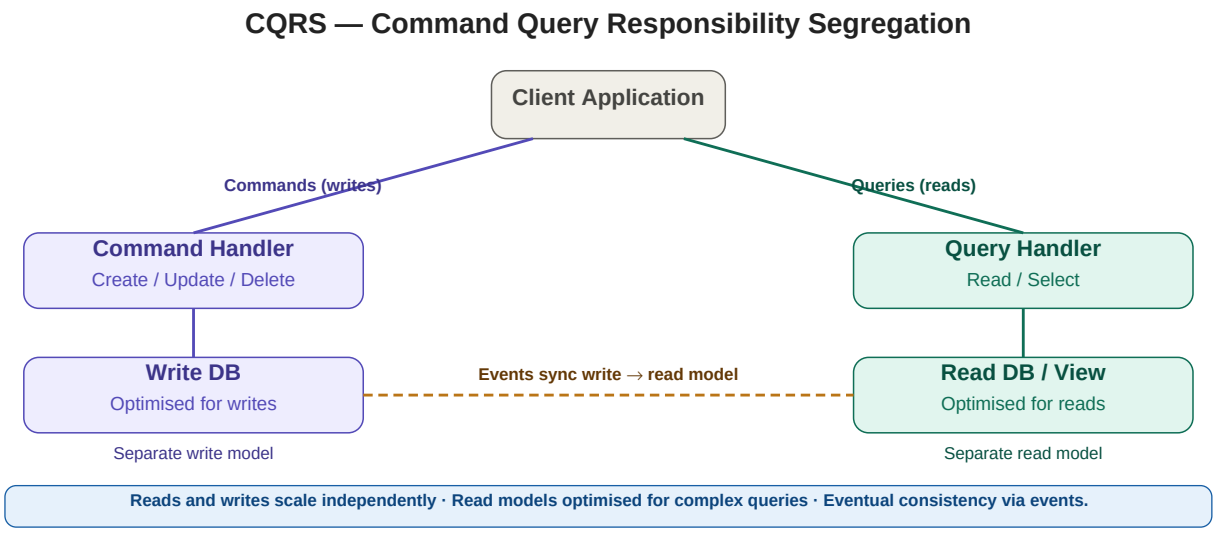


Happy path: events chain Order → Inventory → Payment | Failure path: compensating transactions reverse prior steps

Aspect	Choreography	Orchestration
Control	Decentralised — services react to events	Centralised — orchestrator directs steps
Coupling	Loosely coupled services	Services depend on orchestrator
Visibility	Harder to trace full flow	Clear, easy to reason about
Risk	Complex event chains, circular deps	Orchestrator is a bottleneck
Best for	Simple flows, truly independent services	Complex workflows needing control

8. CQRS — Command Query Responsibility Segregation

CQRS separates write operations (commands: create/update/delete) from read operations (queries: select/read). Each side uses a data model optimised for its workload. Events from the command side asynchronously update the read model.



Commands go to Write DB · Queries go to Read DB · Events keep them in sync

- Solves cross-service join problem — read model aggregates data from multiple services
- Write model optimised for integrity; read model optimised for query performance
- Scales reads and writes independently — common when reads >> writes
- Trade-off: eventual consistency — read model may lag slightly behind write model

9. Quick Revision & Interview Questions

Quick Revision

→ Monolith: single deployable unit, shared DB, tightly coupled — hard to scale, slow to deploy.

→ Microservices: independent services, own DBs, loosely coupled — independently scalable and deployable.

→ Two decomposition patterns: Business Capability (one per function) and DDD (sub-domains within a domain).

→ Strangler Pattern: incremental migration using a router to split traffic between monolith and microservices.

→ Database per service is standard — no cross-service DB access, use events and Saga for coordination.

→ Saga Pattern: sequence of local transactions with compensating transactions on failure.

→ Saga types: Choreography (event-driven, decentralised) vs Orchestration (central controller).

→ CQRS: separates reads and writes — command side writes, query side reads, events keep them in sync.

→ Main microservices challenges: distributed transactions, inter-service latency, complex monitoring.

→ Poor decomposition = distributed monolith — tight coupling, high latency, hard to change independently.

Interview Questions

■ 1. What are the main disadvantages of a monolithic architecture?

■ 2. Why would you choose microservices over a monolith? What problems does it solve?

■ 3. What is the difference between Business Capability and DDD decomposition?

■ 4. How do you migrate a monolith to microservices without downtime? (Strangler Pattern)

■ 5. Why is database-per-service the recommended approach? What are the trade-offs?

■ 6. What is the Saga Pattern? When would you use it over a single ACID transaction?

■ 7. What is the difference between Choreography and Orchestration in the Saga Pattern?

- 8. What is CQRS? How does it solve the cross-service join problem?
- 9. What is a compensating transaction? Give an example.
- 10. What are the challenges of monitoring and debugging in a microservices system?
- 11. How would you design a payment system using microservices with Saga?

Key Terms

Monolith	Single deployable unit — all services in one codebase, one DB
Microservices	Small independent services each owning their own DB and deployment
Business Capability	Decompose by major business function (Orders, Payments, Login)
DDD / Sub-domain	Decompose by sub-domains within a domain (Forward vs Refund Payment)
Strangler Pattern	Incremental migration using a router to gradually shift traffic
DB per Service	Each service owns its own database — no cross-service DB access
Saga Pattern	Distributed transaction via local transactions + compensating rollbacks
Choreography	Saga where services react to events — decentralised
Orchestration	Saga where a central orchestrator directs each step
CQRS	Separates writes (commands) from reads (queries) with separate models
Compensating Txn	Reverse operation that undoes a prior step on transaction failure
Loose Coupling	Services interact via interfaces — change one without breaking others
Eventual Consistency	Data syncs asynchronously — briefly stale but eventually correct